ELDA

Linear Algebra Library Booklet

Mathematical reference, engineering guide, and practical starter manual

---

Dense matrix operations, elimination algorithms, system solving, norms, characteristic polynomials, and homogeneous transforms

---

Document Scope

This booklet documents the current ELDA C++17 library found under `include/elda/` and `src/`. It explains the mathematics used by the library, the engineering structure of the codebase, and the intended usage of every major public API entry point.

Prepared for the project repository

Ayushmaan Kumar Verma
Department of Mathematics and Computing Technology
NIT Patna

April 10, 2026

Preface

ELDA is a compact linear algebra library written in modern C++ and organized as a CMake project. The library exposes a dense matrix type, arithmetic helpers, elimination-based routines, structural invariants such as determinant and rank, and 2D/3D transformation generators based on homogeneous coordinates.

This booklet serves three purposes:

1. It acts as a starter guide for someone who wants to build and use the library quickly.
2. It explains the mathematics that justify each public function, so the code is tied back to linear algebra rather than treated as a black box.
3. It records technical structure and implementation behavior so future maintenance work can stay coherent.

Contents

Preface **i**

1 Library Overview and Starter Guide **1**
1.1 What ELDA provides . . . 1
1.2 Build the project . . . 1
1.3 Minimal usage example . . . 2
1.4 Who this booklet is for . . . 2

2 Mathematical and Technical Foundations **3**
2.1 Notation used in this booklet . . . 3
2.2 Dense storage model . . . 3
2.3 Mutating versus non-mutating operations . . . 3
2.4 Core linear algebra ideas . . . 4
2.4.1 Elementary row operations . . . 4
2.4.2 Homogeneous coordinates . . . 4

3 Matrix Type, Constructors, and Basic API **5**
3.1 Constructors . . . 5
3.1.1 `matrix()` . . . 5
3.1.2 `matrix(int r, int c)` . . . 5
3.2 Element access and I/O . . . 6
3.2.1 `get_element(i, j)` . . . 6
3.2.2 `ref_element(i, j)` . . . 6
3.2.3 `input()` . . . 6
3.2.4 `print()` . . . 6
3.3 Assignment and comparisons . . . 6
3.3.1 `operator=(matrix m2)` . . . 6
3.3.2 `operator==(matrix m1, matrix m2)` . . . 7
3.3.3 `shape_comp(matrix m1, matrix m2)` . . . 7
3.4 Identity and structure helpers . . . 7
3.4.1 `identity(int n)` . . . 7
3.4.2 `get_row(int r)` . . . 7
3.4.3 `get_col(int c)` . . . 8

4 Arithmetic Operations **9**
4.1 `operator+(matrix m2)` . . . 9
4.2 `operator-(matrix m2)` . . . 9
4.3 `operator*(double d)` . . . 9
4.4 `operator*(matrix m2)` . . . 10

5 Elementary Row and Column Operations **11**
5.1 `row_op(int i, double coeff, int j)` . . . 11
5.2 `col_op(int i, double coeff, int j)` . . . 11
5.3 `row_swap(int i, int j)` and `col_swap(int i, int j)` . . . 12
5.4 `row_multi(int i, double factor)` and `col_multi(int i, double factor)` . 12

6 Elimination, Rank, and Linear Systems **13**
6.1 `echelon()` . . . 13
6.2 `gaussian()` . . . 13
6.3 `gauss_jordan()` . . . 14
6.4 `canonical()` . . . 15
6.5 `rank()` . . . 15
6.6 `free_variable()` . . . 15
6.7 `solve()` . . . 16

7 Determinants, Cofactors, and Inverses **17**
7.1 `trace()` . . . 17
7.2 `diag_product()` . . . 17
7.3 `det()` . . . 17
7.4 `minor(int x, int y)` . . . 18
7.5 `cofactor(int x, int y)` . . . 18
7.6 `adjoint()` . . . 19
7.7 `transpose()` . . . 19
7.8 `inverse()` . . . 19

8 Norms, Inner Products, Matrix Powers, and Characteristic Polynomials **21**
8.1 `norm()` . . . 21
8.2 `inner_product(matrix a, matrix b)` . . . 21
8.3 `angle(matrix a, matrix b)` . . . 21
8.4 `matpow(matrix mat, long long expo)` . . . 22
8.5 `char_poly()` . . . 22
8.6 `check_ortho(matrix mat)` . . . 23
8.7 `check_unitary(matrix mat)` . . . 23

9 Transformation Helpers **24**
9.1 2D transformation matrices . . . 24
9.1.1 `shift(double dx, double dy)` . . . 24
9.1.2 `scale(double kx, double ky)` . . . 24
9.1.3 `rotate(double angle)` . . . 24
9.2 3D transformation matrices . . . 25
9.2.1 `shift(double dx, double dy, double dz)` . . . 25
9.2.2 `scale(double kx, double ky, double kz)` . . . 25
9.2.3 `rot_x(double angle)` . . . 25
9.2.4 `rot_y(double angle)` . . . 25
9.2.5 `rot_z(double angle)` . . . 26

10 Library-Specific Utilities and Numerical Notes **27**
10.1 `vec1()` to `vec5()` and value overloads . . . 27
10.2 `check_lin_comb(...)` . . . 27
10.3 `neg_zero(matrix\& m)` . . . 28
10.4 Constants . . . 28
10.5 Numerical caveats . . . 28

10.6 Shape prerequisites summary . . . 28

11 Engineering Structure and Maintenance Notes **29**
11.1 Header organization . . . 29
11.2 Implementation organization . . . 29
11.3 Mutability policy . . . 29
11.4 Starter checklist for contributors . . . 29

A Quick Reference **31**
A.1 Public API map . . . 31

B Build This Booklet **33**
B.1 Command line . . . 33
B.2 Why this format works well . . . 33

C Complete 80-Function Catalog **34**
C.1 Scope . . . 34
C.2 Matrix Construction and Access . . . 34
C.2.1 `matrix()` . . . 34
C.2.2 `matrix(int r, int c)` . . . 34
C.2.3 `matrix(const matrix\&)` . . . 34
C.2.4 `get_element(int i, int j)` . . . 35
C.2.5 `ref_element(int i, int j)` . . . 35
C.2.6 `input()` . . . 35
C.2.7 `print()` . . . 35
C.3 Arithmetic and Comparison . . . 35
C.3.1 `operator=(matrix)` . . . 35
C.3.2 `operator+(matrix)` . . . 35
C.3.3 `operator-(matrix)` . . . 36
C.3.4 `operator*(matrix)` . . . 36
C.3.5 `operator*(double)` . . . 36
C.3.6 `operator==(matrix, matrix)` . . . 36
C.3.7 `shape\_comp(matrix, matrix)` . . . 36
C.3.8 `identity(int n)` . . . 37
C.4 Elementary Row and Column Operations . . . 37
C.4.1 `row\_op(int i, double coeff, int j)` . . . 37
C.4.2 `col\_op(int i, double coeff, int j)` . . . 37
C.4.3 `row\_swap(int i, int j)` . . . 37
C.4.4 `col\_swap(int i, int j)` . . . 37
C.4.5 `row\_multi(int i, double factor)` . . . 37
C.4.6 `col\_multi(int i, double factor)` . . . 38
C.4.7 `get\_row(int r)` . . . 38
C.4.8 `get\_col(int c)` . . . 38
C.4.9 `get\_row\_vec(int r)` . . . 38
C.4.10 `get\_col\_vec(int c)` . . . 38
C.4.11 `replace\_row(int r, matrix rw)` . . . 38
C.4.12 `replace\_col(int c, matrix cn)` . . . 39
C.5 Elimination and System Solving . . . 39
C.5.1 `echelon()` . . . 39
C.5.2 `gaussian()` . . . 39
C.5.3 `gauss\_jordan()` . . . 39
C.5.4 `canonical()` . . . 39

C.5.5 rank() . . . 39
C.5.6 free_variable() . . . 40
C.5.7 solve() . . . 40
C.6 Matrix Invariants and Derived Matrices . . . 40
C.6.1 trace() . . . 40
C.6.2 diag_product() . . . 40
C.6.3 det() . . . 40
C.6.4 minor(int x, int y) . . . 40
C.6.5 cofactor(int x, int y) . . . 41
C.6.6 adjoint() . . . 41
C.6.7 transpose() . . . 41
C.6.8 inverse() . . . 41
C.6.9 orthogonalize() . . . 41
C.6.10 orthonormalize() . . . 41
C.6.11 qr_decomp_q() . . . 42
C.6.12 qr_decomp_r() . . . 42
C.6.13 check_upper_tri() . . . 42
C.6.14 check_lower_tri() . . . 42
C.6.15 char_poly() . . . 42
C.6.16 eigenvalues() . . . 42
C.6.17 norm() . . . 43
C.7 Global Numerical and Algebra Helpers . . . 43
C.7.1 neg_zero(matrix\& m) . . . 43
C.7.2 fpg(matrix\& m) . . . 43
C.7.3 matpow(matrix mat, long long expo) . . . 43
C.7.4 check_ortho(matrix mat) . . . 43
C.7.5 check_unitary(matrix mat) . . . 43
C.7.6 inner_product(matrix a, matrix b) . . . 44
C.7.7 angle(matrix a, matrix b) . . . 44
C.8 Vector Utility Constructors and Span Checks . . . 44
C.8.1 vec1() . . . 44
C.8.2 vec2() . . . 44
C.8.3 vec3() . . . 44
C.8.4 vec4() . . . 44
C.8.5 vec5() . . . 45
C.8.6 vec1(double x) . . . 45
C.8.7 vec2(double x, double y) . . . 45
C.8.8 vec3(double x, double y, double z) . . . 45
C.8.9 vec4(double x, double y, double z, double w) . . . 45
C.8.10 vec5(double x, double y, double z, double w, double u) . . . 45
C.8.11 check_lin_comb(matrix m1, matrix m2) . . . 45
C.8.12 check_lin_comb(matrix m1, matrix m2, matrix m3) . . . 46
C.8.13 check_lin_comb(matrix m1, matrix m2, matrix m3, matrix m4) . . . 46
C.8.14 check_lin_comb(matrix m1, matrix m2, matrix m3, matrix m4, matrix m5) . . . 46
C.9 2D and 3D Transformation Factories . . . 46
C.9.1 shift(double dx, double dy) . . . 46
C.9.2 scale(double kx, double ky) . . . 46
C.9.3 rotate(double angle) . . . 47
C.9.4 shift(double dx, double dy, double dz) . . . 47
C.9.5 scale(double kx, double ky, double kz) . . . 47

C.9.6 `rot\_x(double angle)` 47
C.9.7 `rot\_y(double angle)` 47
C.9.8 `rot\_z(double angle)` 47

Closing Note **48**

1 Library Overview and Starter Guide

1.1 What ELDA provides

At its core, ELDA provides a `linalg::matrix` type whose entries are stored as `double` values in a row-major `std::vector<std::vector<double>>`. The public API is grouped into three headers:

- `elda/matrix.hpp`: the matrix type and core algebraic routines,
- `elda/transforms.hpp`: 2D and 3D homogeneous transformation matrices,
- `elda/vector_utils.hpp`: vector constructors (`vec1` to `vec5`) and linear-combination checks.

Project Layout

Path	Purpose
`CMakeLists.txt`	Defines the static library target `elda` and the demo executable `main`.
`include/elda/matrix.hpp`	Public declaration of the matrix type and most free functions.
`include/elda/transforms.hpp`	Public declaration of 2D and 3D transform helpers.
`include/elda/vector_utils.hpp`	Public declaration of `vec1`-`vec5` constructors and `check_lin_comb` overloads.
`include/elda/linalg.hpp`	Umbrella header that includes the whole public API.
`src/matrix.cpp`	Implementation of matrix arithmetic, elimination, determinant, inverse, and spectral helpers.
`src/transforms.cpp`	Implementation of translation, scaling, and rotation matrices.
`src/vector_utils.cpp`	Implementation of vector constructors and span-membership helpers.
`main.cpp`	Small demo program that prints a sample matrix and its eigenvalue estimates.

1.2 Build the project

The library is configured with CMake and targets C++17.

Listing 1.1: Build commands

```
cmake -S . -B build
cmake --build build
```

After a successful build, the main artifacts are:

- `build/libelda.a`: the static library,
- `build/main`: the demo executable.

1.3 Minimal usage example

The fastest way to start is to include the umbrella header and work with a small dense matrix.

Listing 1.2: Minimal ELDA program

```
#include <elda/linalg.hpp>
#include <iostream>

using namespace linalg;

int main() {
    matrix a(2, 2);
    a.arr = {{1, 2}, {3, 4}};

    matrix b = identity(2);
    matrix c = a + b;

    c.print();
}
```

Starter interpretation

The matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

is added to the identity matrix

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

which yields

$$A + I_2 = \begin{bmatrix} 2 & 2 \\ 3 & 5 \end{bmatrix}.$$

This is entrywise matrix addition, so each diagonal entry increases by one.

1.4 Who this booklet is for

This document is written for three audiences:

- users who want to apply the library correctly,
- maintainers who need to understand why an algorithm exists,
- learners who want the code tied back to linear algebra concepts.

2

Mathematical and Technical Foundations

2.1 Notation used in this booklet

Unless stated otherwise, a matrix

$$A = (a_{ij}) \in \mathbb{R}^{m \times n}$$

has m rows and n columns. In the C++ code, the dimensions are stored in the public members `row` and `col`, and indexing is zero-based because the underlying storage is a C++ vector.

Indexing convention

Mathematics often writes a_{11} for the first element of a matrix. ELDA stores that same element at `arr[0][0]`. In formulas below we use the standard mathematical notation, but examples mention the corresponding C++ indices whenever confusion is possible.

2.2 Dense storage model

The matrix class stores data as

```
std::vector<std::vector<double>>.
```

This is easy to inspect, easy to print, and convenient for educational code. The tradeoff is that it is not as cache-friendly as a flat contiguous buffer. For the scale of a teaching-oriented or small library, the design is reasonable and transparent.

2.3 Mutating versus non-mutating operations

ELDA has two styles of API:

- **return-a-copy helpers**: arithmetic operators, row and column operations, transpose, inverse, adjoint, and transform factories return new matrices;
- **in-place reducers**: `echelon()`, `gaussian()`, `gauss_jordan()`, and `canonical()` mutate the matrix object and return an integer that records the sign change induced by row swaps.

2.4 Core linear algebra ideas

2.4.1 Elementary row operations

Three row operations preserve the solution set of a linear system:

1. swap two rows,
2. multiply a row by a nonzero scalar,
3. replace one row by itself plus a multiple of another row.

These operations are the engine behind Gaussian elimination, rank computation, determinant reduction, and matrix inversion. ELDA also exposes column analogues of the same operations.

2.4.2 Homogeneous coordinates

The transformation helpers in `transforms.hpp` use homogeneous coordinates. A 2D point (x, y) is encoded as

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

and a 3D point (x, y, z) is encoded as

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}.$$

This lets translation, scaling, and rotation all be represented as matrix multiplication.

3 Matrix Type, Constructors, and Basic API

3.1 Constructors

3.1.1 `matrix()`

The default constructor builds a 3×3 zero matrix.

Mathematical meaning

The zero matrix $0_{3\times 3}$ is the additive identity in the vector space of 3×3 real matrices. For every $A \in \mathbb{R}^{3\times 3}$,

$$A + 0_{3\times 3} = A.$$

Listing 3.1: Default constructor example

```
matrix z;
z.print();
```

Example

The code above prints a 3×3 grid of zeros.

3.1.2 `matrix(int r, int c)`

This constructor builds the zero matrix $0_{r\times c}$.

Listing 3.2: Sized constructor example

```
matrix a(2, 3);
a.arr = {{1, 2, 3}, {4, 5, 6}};
```

Example

Before assigning entries, `a` is

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

3.2 Element access and I/O

3.2.1 `get_element(i, j)`

This returns the entry a_{ij} with no bounds check. The mathematics is direct: it selects one coordinate of the matrix.

Example

If

$$A = \begin{bmatrix} 7 & 8 \\ 9 & 10 \end{bmatrix},$$

then `get_element(1, 0)` returns 9, because C++ index $(1, 0)$ is the second row, first column.

3.2.2 `ref_element(i, j)`

This returns a mutable pointer to the selected element. In mathematical terms it gives write access to one coordinate of the matrix.

Listing 3.3: Pointer-based update

```
matrix a(2, 2);
*a.ref_element(0, 1) = 5.0;
```

3.2.3 `input()`

This reads all entries row by row from standard input. Mathematically it populates a matrix by specifying the coordinates a_{ij} in row-major order.

3.2.4 `print()`

This writes the matrix one row per line. It is a presentation helper rather than a mathematical operation.

3.3 Assignment and comparisons

3.3.1 `operator=(matrix m2)`

The library defines assignment as a shape-preserving copy. If the dimensions of the left-hand side and right-hand side differ, an exception is thrown.

Important implementation note

Standard C++ assignment usually resizes the destination object when needed. ELDA's current assignment operator instead requires equal dimensions. That is a library design decision, not a language requirement.

Listing 3.4: Assignment example

```
matrix a(2, 2), b(2, 2);
b.arr = {{1, 0}, {0, 1}};
a = b;
```

Example

After assignment, $A = B = I_2$.

3.3.2 `operator==(matrix m1, matrix m2)`

This function checks exact entrywise equality:

$$A = B \quad \Longleftrightarrow \quad a_{ij} = b_{ij} \text{ for all } i, j.$$

Example

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

so the comparison returns `true`.

3.3.3 `shape_comp(matrix m1, matrix m2)`

This tests only the dimensions. It corresponds to the statement $A, B \in \mathbb{R}^{m \times n}$ for the same m, n.

Example

A 2×3 matrix and another 2×3 matrix are shape-compatible even if their entries are different.

3.4 Identity and structure helpers

3.4.1 `identity(int n)`

This builds the identity matrix

$$I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}.$$

Why it matters

The identity matrix is the multiplicative identity:

$$AI_n = A, \qquad I_n A = A.$$

It is central to inversion, matrix powers, and transform construction.

Example

`identity(3)` returns I_3, the matrix with ones on the main diagonal and zeros elsewhere.

3.4.2 `get_row(int r)`

This returns a matrix with row r preserved and all other rows set to zero. If A is $m \times n$, the result is also $m \times n$, not a $1 \times n$ row vector.

Example

If

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

then `get_row(1)` returns

$$\begin{bmatrix} 0 & 0 \\ 3 & 4 \end{bmatrix}.$$

3.4.3 `get_col(int c)`

This returns a matrix with column c preserved and all other columns set to zero.

Example

For the same matrix A,

$$\texttt{get_col(0)} = \begin{bmatrix} 1 & 0 \\ 3 & 0 \end{bmatrix}.$$

4 Arithmetic Operations

4.1 `operator+(matrix m2)`

Matrix addition is defined entrywise:

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

The operation is valid only when both matrices have the same dimensions.

Example

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}.$$

4.2 `operator-(matrix m2)`

Subtraction is also entrywise:

$$(A - B)_{ij} = a_{ij} - b_{ij}.$$

Example

$$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}.$$

4.3 `operator*(double d)`

Scalar multiplication rescales every entry:

$$(dA)_{ij} = d \cdot a_{ij}.$$

Example

$$2 \begin{bmatrix} 1 & -1 \\ 3 & 0 \end{bmatrix} = \begin{bmatrix} 2 & -2 \\ 6 & 0 \end{bmatrix}.$$

In ELDA this is written as `A * 2.0`.

4.4 `operator*(matrix m2)`

Matrix multiplication is defined by row-column pairing:

$$(AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj},$$

where $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. The result lies in $\mathbb{R}^{m \times p}$.

Interpretation

The entry (i, j) of AB is the dot product of row i of A with column j of B. This is why the inner dimensions must match.

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}.$$

Then

$$AB = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 1 & 1 \cdot 0 + 2 \cdot 5 \\ 3 \cdot 2 + 4 \cdot 1 & 3 \cdot 0 + 4 \cdot 5 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 20 \end{bmatrix}.$$

5 Elementary Row and Column Operations

5.1 `row_op(int i, double coeff, int j)`

This performs the elementary row replacement

$$R_i \leftarrow R_i + \text{coeff} \cdot R_j.$$

Why this matters

Row replacement preserves the solution set of a linear system and preserves the determinant. It is the workhorse operation in elimination.

Example

Start with

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

Applying `row_op(1, -3, 0)` means

$$R_2 \leftarrow R_2 - 3R_1,$$

so the result is

$$\begin{bmatrix} 1 & 2 \\ 0 & -2 \end{bmatrix}.$$

5.2 `col_op(int i, double coeff, int j)`

This is the column analogue:

$$C_i \leftarrow C_i + \text{coeff} \cdot C_j.$$

Column operations are useful in canonical forms and matrix equivalence arguments.

Example

For

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

applying `col_op(1, -2, 0)` gives

$$C_2 \leftarrow C_2 - 2C_1, \qquad \begin{bmatrix} 1 & 0 \\ 3 & -2 \end{bmatrix}.$$

5.3 `row_swap(int i, int j)` and `col_swap(int i, int j)`

Swapping rows or columns permutes the matrix.

Determinant effect

Swapping two rows multiplies the determinant by -1. Swapping two columns does the same. This sign flip is why the elimination routines keep track of swap parity.

Example

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \xrightarrow{\text{swap rows}} \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}.$$

5.4 `row_multi(int i, double factor)` and `col_multi(int i, double factor)`

These scale one row or one column:

$$R_i \leftarrow \lambda R_i, \qquad C_j \leftarrow \lambda C_j.$$

Determinant effect

If a row of a square matrix is multiplied by λ, then the determinant is multiplied by λ. Gaussian elimination uses row scaling to normalize pivots.

Example

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \xrightarrow{R_1 \leftarrow 2R_1} \begin{bmatrix} 2 & 4 \\ 3 & 4 \end{bmatrix}.$$

6 Elimination, Rank, and Linear Systems

6.1 `echelon()`

This mutates the matrix into row-echelon form by searching for a nonzero pivot in each column, swapping that row upward if necessary, and eliminating entries below the pivot.

Row-echelon form

A matrix is in row-echelon form when:

1. every nonzero row starts with a leading nonzero entry called a pivot,
2. pivot positions move strictly to the right as you move downward,
3. entries below each pivot are zero.

Example

Let

$$A = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}.$$

After `echelon()`, ELDA performs

$$R_2 \leftarrow R_2 - 2R_1,$$

which produces

$$\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}.$$

This is already in row-echelon form.

6.2 `gaussian()`

This is a normalized Gaussian elimination. It first creates a pivot, then scales the pivot row so the pivot becomes 1, and finally clears entries below it.

Gaussian elimination with normalized pivots

For pivot position (k, k), ELDA applies

$$R_k \leftarrow \frac{1}{a_{kk}} R_k$$

when $a_{kk} \neq 0$, then uses row replacement to zero all entries below the pivot. The resulting form is upper row-echelon with leading ones whenever possible.

Example

For

$$\begin{bmatrix} 3 & 6 \\ 6 & 15 \end{bmatrix},$$

the first pivot row is scaled by $1/3$, giving

$$\begin{bmatrix} 1 & 2 \\ 6 & 15 \end{bmatrix},$$

then $R_2 \leftarrow R_2 - 6R_1$, yielding

$$\begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix},$$

and the second pivot row is scaled to obtain

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}.$$

6.3 `gauss_jordan()`

This calls `gaussian()` first, then eliminates the entries above each pivot to obtain reduced row-echelon form.

Reduced row-echelon form

A matrix is in reduced row-echelon form when it is in row-echelon form and every pivot column contains a single 1 with zeros everywhere else.

Example

Starting from

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix},$$

the routine clears the entry above the second pivot:

$$R_1 \leftarrow R_1 - 2R_2,$$

producing

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

6.4 canonical()

This routine first runs normalized Gaussian elimination, then applies column operations to clear entries to the right of each pivot. The effect is a mixed row-column reduction that pushes the matrix toward a diagonal or canonical pattern.

Conceptual role

Row operations preserve the solution set of a linear system, while column operations change the coordinate representation of the domain. Combining them is natural when studying matrix equivalence or canonical forms, rather than only solving systems.

Example

If Gaussian elimination has already produced

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix},$$

then a column operation

$$C_2 \leftarrow C_2 - 2C_1$$

transforms it into the identity matrix. This is the kind of diagonal-cleaning behavior that `canonical()` performs.

6.5 rank()

The rank of a matrix is the dimension of its row space, equivalently the number of pivots in row-echelon form:

$$\operatorname{rank}(A) = \text{number of nonzero rows in an echelon form of } A.$$

ELDA computes this by applying `gaussian()` to a copy and counting nonzero rows.

Example

For

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix},$$

the second row is a multiple of the first, so there is only one pivot. Hence $\operatorname{rank}(A) = 1$.

6.6 free_variable()

For a square coefficient matrix $A \in \mathbb{R}^{n \times n}$, this returns

$$n - \operatorname{rank}(A).$$

This matches the nullity of A for square systems, by the rank-nullity theorem.

Example

If a 3×3 matrix has rank 2, then the routine returns $3 - 2 = 1$, meaning one free variable in the homogeneous system $Ax = 0$.

6.7 `solve()`

This function expects an augmented matrix of size $n \times (n+1)$:

$$\left[\begin{array}{c|c} A & b \end{array}\right].$$

It runs normalized Gaussian elimination and then performs back substitution to produce the solution vector.

Back substitution

If the reduced system is upper triangular,

$$\begin{aligned} x_n &= b_n, \\ x_{n-1} &= b_{n-1} - a_{n-1,n}x_n, \\ x_{n-2} &= b_{n-2} - a_{n-2,n-1}x_{n-1} - a_{n-2,n}x_n, \end{aligned}$$

and so on upward.

Example

Solve

$$\begin{cases} x + y = 3, \\ 2x + 3y = 8. \end{cases}$$

The augmented matrix is

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 2 & 3 & 8 \end{array}\right].$$

Gaussian elimination gives

$$\left[\begin{array}{cc|c} 1 & 1 & 3 \\ 0 & 1 & 2 \end{array}\right].$$

Back substitution yields $y = 2$ and $x = 1$.

7
Determinants, Cofactors, and Inverses

7.1 `trace()`

The trace is the sum of diagonal entries:

$$\operatorname{tr}(A) = \sum_{i=1}^{\min(m,n)} a_{ii}.$$

For square matrices, the trace equals the sum of eigenvalues counted with algebraic multiplicity.

Example

$$\operatorname{tr}\left(\begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}\right) = 2 + 3 = 5.$$

7.2 `diag_product()`

This returns the product of the diagonal entries:

$$\prod_{i=1}^{\min(m,n)} a_{ii}.$$

For triangular square matrices, this product is exactly the determinant.

Example

For

$$\begin{bmatrix} 2 & 7 \\ 0 & 5 \end{bmatrix},$$

`diag_product()` returns $2 \cdot 5 = 10$.

7.3 `det()`

The determinant of a square matrix is computed by converting a copy of the matrix to row-echelon form and then multiplying the diagonal entries, adjusted by the sign of row swaps.

Why elimination works

Two determinant facts make the algorithm valid:

1. replacing one row by itself plus a multiple of another row leaves the determinant unchanged,
2. swapping two rows changes the sign of the determinant.

Once the matrix is upper triangular,

$$\det(A) = \prod_i u_{ii}.$$

Example

For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

perform

$$R_2 \leftarrow R_2 - 3R_1$$

to get

$$\begin{bmatrix} 1 & 2 \\ 0 & -2 \end{bmatrix}.$$

No row swap occurred, so

$$\det(A) = 1 \cdot (-2) = -2.$$

7.4 `minor(int x, int y)`

The minor of entry (x, y) is the determinant of the submatrix obtained by deleting row x and column y.

Example

For

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix},$$

the minor of the center entry is

$$\det \begin{bmatrix} 1 & 3 \\ 7 & 9 \end{bmatrix} = 9 - 21 = -12.$$

7.5 `cofactor(int x, int y)`

The cofactor adds the alternating sign pattern:

$$C_{xy} = (-1)^{x+y} M_{xy},$$

where M_{xy} is the minor.

Example

If the minor at $(1,1)$ equals -12, then the cofactor is

$$(-1)^{1+1}(-12) = -12.$$

If the same minor were at $(1,2)$, the cofactor would flip sign.

7.6 adjoint()

The classical adjoint, also called the adjugate, is the transpose of the cofactor matrix:

$$\operatorname{adj}(A) = C^\top.$$

Inverse connection

For a nonsingular square matrix,

$$A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A).$$

ELDA does not use this identity to compute the inverse, but the formula explains why the adjoint is mathematically important.

Example

For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

the adjugate is

$$\operatorname{adj}(A) = \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}.$$

7.7 transpose()

The transpose exchanges rows and columns:

$$(A^\top)_{ij} = a_{ji}.$$

It is fundamental for symmetric matrices, orthogonality checks, and the Frobenius inner product.

Example

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

7.8 inverse()

The inverse A^{-1} of a square matrix satisfies

$$AA^{-1} = A^{-1}A = I.$$

ELDA computes it through Gauss-Jordan elimination on the augmented block matrix

$$\left[\begin{array}{c|c} A & I \end{array}\right].$$

When the left block is reduced to I, the right block becomes A^{-1}.

Example

For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

augment with the identity:

$$\left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array}\right].$$

After Gauss-Jordan elimination, the result is

$$\left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & \frac{3}{2} & -\frac{1}{2} \end{array}\right],$$

so

$$A^{-1} = \begin{bmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{bmatrix}.$$

8

Norms, Inner Products, Matrix Powers, and Characteristic Polynomials

8.1 `norm()`

ELDA uses the Frobenius norm:

$$\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2}.$$

This is the Euclidean norm of the matrix entries viewed as one long vector.

Example

For

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix},$$

$$\|A\|_F = \sqrt{1^2 + 2^2 + 2^2 + 1^2} = \sqrt{10}.$$

8.2 `inner_product(matrix a, matrix b)`

The function computes the Frobenius inner product:

$$\langle A, B \rangle_F = \operatorname{tr}(A^\top B) = \sum_{i,j} a_{ij} b_{ij}.$$

Example

If

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 & 4 \\ 5 & 6 \end{bmatrix},$$

then

$$\langle A, B \rangle_F = 1 \cdot 3 + 2 \cdot 4 + 0 \cdot 5 + 1 \cdot 6 = 17.$$

8.3 `angle(matrix a, matrix b)`

The angle between two matrices is defined through the Frobenius inner product:

$$\theta = \cos^{-1}\left(\frac{\langle A, B \rangle_F}{\|A\|_F \|B\|_F}\right).$$

This generalizes the Euclidean angle between vectors.

Example

If $A = I_2$ and $B = I_2$, then the numerator equals the product of the norms, so the cosine is 1 and the angle is 0.

8.4 matpow(matrix mat, long long expo)

This computes A^k for a nonnegative integer k using repeated squaring.

Repeated squaring

Instead of multiplying A by itself k times in sequence, repeated squaring uses the binary expansion of k. For example,

$$A^{13} = A^8 A^4 A.$$

This reduces the number of matrix multiplications from $O(k)$ to $O(\log k)$.

Example

For

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix},$$

one gets

$$A^2 = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \qquad A^3 = \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}.$$

8.5 char_poly()

This routine implements the Faddeev-LeVerrier recurrence to compute the coefficients of the characteristic polynomial

$$p_A(\lambda) = \det(\lambda I - A) = \lambda^n + c_1\lambda^{n-1} + \cdots + c_n$$

for an $n \times n$ matrix.

Faddeev-LeVerrier recurrence

The algorithm used by ELDA follows the pattern

$$B_0 = I, \qquad c_k = -\frac{1}{k}\operatorname{tr}(AB_{k-1}), \qquad B_k = AB_{k-1} + c_k I.$$

Each c_k becomes one coefficient of the characteristic polynomial.

Example

For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

the characteristic polynomial is

$$\lambda^2 - 5\lambda - 2.$$

So the returned row vector is

$$\begin{bmatrix} 1 & -5 & -2 \end{bmatrix}.$$

8.6 check_ortho(matrix mat)

An orthogonal real matrix satisfies

$$A^\top A = I,$$

which is equivalent to

$$A^{-1} = A^\top.$$

ELDA checks the second formulation by comparing the transpose and inverse directly.

Example

The planar rotation matrix

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

is orthogonal for all real θ.

8.7 check_unitary(matrix mat)

In standard linear algebra, a complex matrix is unitary when

$$U^* U = I,$$

where U^* is the conjugate transpose. ELDA's current helper compares `transpose()` and `adjoint()`, where `adjoint()` means the classical adjugate, not the conjugate transpose.

Important terminology note

This means `check_unitary()` should be understood as a legacy project helper rather than a standard mathematical unitary test.

Example

For the identity matrix, the transpose and adjugate are both equal to the identity, so the helper returns `true`. That does not validate the helper as a complete unitary predicate in the usual mathematical sense.

9

Transformation Helpers

9.1 2D transformation matrices

9.1.1 `shift(double dx, double dy)`

This returns the homogeneous translation matrix

$$T(dx, dy) = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix}.$$

Example

Applying this matrix to the homogeneous point $(x, y, 1)^\top$ gives

$$\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + dx \\ y + dy \\ 1 \end{bmatrix}.$$

9.1.2 `scale(double kx, double ky)`

This returns

$$S(k_x, k_y) = \begin{bmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

which scales the x-coordinate by k_x and the y-coordinate by k_y.

Example

If $(x, y) = (2, 3)$ and $(k_x, k_y) = (4, 5)$, then the scaled point is $(8, 15)$.

9.1.3 `rotate(double angle)`

This returns the standard planar rotation matrix in homogeneous form:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Angles are measured in radians. The header also defines the convenience constant `PI = 3.141593`.

Example

When $\theta = \pi/2$,

$$R(\pi/2)\begin{bmatrix}1\\0\\1\end{bmatrix} = \begin{bmatrix}0\\1\\1\end{bmatrix},$$

so the point $(1, 0)$ rotates to $(0, 1)$.

9.2 3D transformation matrices

9.2.1 `shift(double dx, double dy, double dz)`

This returns

$$\begin{bmatrix}1 & 0 & 0 & dx\\0 & 1 & 0 & dy\\0 & 0 & 1 & dz\\0 & 0 & 0 & 1\end{bmatrix}.$$

It translates points in $\mathbb{R}^3$ by the vector (dx, dy, dz).

9.2.2 `scale(double kx, double ky, double kz)`

This returns the diagonal homogeneous scaling matrix

$$\begin{bmatrix}k_x & 0 & 0 & 0\\0 & k_y & 0 & 0\\0 & 0 & k_z & 0\\0 & 0 & 0 & 1\end{bmatrix}.$$

9.2.3 `rot_x(double angle)`

Rotation about the x-axis is

$$R_x(\theta) = \begin{bmatrix}1 & 0 & 0 & 0\\0 & \cos\theta & -\sin\theta & 0\\0 & \sin\theta & \cos\theta & 0\\0 & 0 & 0 & 1\end{bmatrix}.$$

9.2.4 `rot_y(double angle)`

Rotation about the y-axis is

$$R_y(\theta) = \begin{bmatrix}\cos\theta & 0 & -\sin\theta & 0\\0 & 1 & 0 & 0\\\sin\theta & 0 & \cos\theta & 0\\0 & 0 & 0 & 1\end{bmatrix}.$$

9.2.5 `rot_z(double angle)`

Rotation about the z-axis is

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Composite transform example

Suppose a point $p = (1, 2, 3)$ is represented as $(1, 2, 3, 1)^\top$. If you first rotate around the z-axis and then translate by $(5, 0, -1)$, the combined transform is

$$T(5, 0, -1)R_z(\theta).$$

Matrix multiplication composes transformations from right to left when column vectors are used.

10 Library-Specific Utilities and Numerical Notes

10.1 `vec1()` to `vec5()` and value overloads

The vector utility module provides convenience constructors for small column vectors:

- zero-initialized constructors: `vec1()`, `vec2()`, `vec3()`, `vec4()`, `vec5()`,
- value constructors: `vec1(x)`, `vec2(x,y)`, ..., `vec5(x,y,z,w,u)`.

Each helper returns an $n \times 1$ matrix, so vectors can be used directly with matrix multiplication and transformation helpers.

Example

```
matrix point = vec4(1.0, 0.0, 0.0, 1.0);
matrix zero3 = vec3();
```

10.2 `check_lin_comb(...)`

The `check_lin_comb` overloads test whether the last input vector lies in the span of earlier vectors. For example:

- `check_lin_comb(m1, m2)` tests whether $m_2 \in \text{span}\{m_1\}$,
- `check_lin_comb(m1, m2, m3)` tests whether $m_3 \in \text{span}\{m_1, m_2\}$, and so on up to five vectors.

The implementation uses a rank comparison of column-assembled matrices.

Input expectations

These overloads are intended for compatible column vectors and do not perform explicit shape validation.

Example

```
bool in_span = check_lin_comb(vec2(1.0, 0.0), vec2(2.0, 0.0));
```

The result is `true` because $(2,0)^\top = 2(1,0)^\top$.

10.3 `neg_zero(matrix\& m)`

This is a display cleanup helper that replaces -0 with 0. The mathematics is trivial because $-0 = 0$, but floating-point output sometimes preserves the signed zero bit.

10.4 Constants

The header defines two public constants:

- `PI = 3.141593`, used naturally in trigonometric transform examples,
- `EPS = 1e-6`, used as the near-zero threshold in `fpg()`.

The cleanup helper `fpg()` maps entries with $|x| \leq$ `EPS` to zero.

10.5 Numerical caveats

The current library uses exact comparisons against zero in several places. That is simple and readable, but in floating-point numerical linear algebra it can be fragile.

Practical warning

If a pivot is extremely small rather than exactly zero, elimination without tolerance checks can amplify roundoff error. `EPS` is used by `fpg()` for cleanup, but many algorithmic branch checks still use exact comparisons.

10.6 Shape prerequisites summary

Function family	Required shape or condition
`operator+`, `operator-`, `operator==`, `shape_comp`	Same dimensions
`operator*(matrix)`	Left columns = right rows
`det`, `minor`, `cofactor`, `adjoint`, `inverse`	Square matrix
`eigenvalues`	Square matrix
`solve`	Augmented matrix of shape $n \times (n+1)$
`free_variable`	Current implementation expects a square coefficient matrix
`char_poly`	Mathematically intended for square matrices

11 Engineering Structure and Maintenance Notes

11.1 Header organization

- `matrix.hpp` centralizes both the class interface and several free functions.
- `transforms.hpp` keeps geometry-related factories separate from algebraic routines.
- `linalg.hpp` provides one include point for consumers who want the entire API.

11.2 Implementation organization

The implementation is split by responsibility:

- `src/matrix.cpp` handles the heavy mathematical work,
- `src/transforms.cpp` stays focused on transformation matrix construction,
- `src/vector_utils.cpp` contains small vector constructors and linear-combination helpers.

11.3 Mutability policy

One of the main design ideas in ELDA is the distinction between:

- operations that conceptually create a new matrix, such as $A + B$, $A^\top$, or A^{-1},
- operations that conceptually reduce a matrix in place, such as Gaussian elimination.

This policy keeps the user-facing code readable, but maintainers should be explicit in documentation about which functions mutate state.

11.4 Starter checklist for contributors

1. Build the project with CMake and run the demo executable.
2. Read `include/elda/matrix.hpp` and map each declaration to its implementation in `src/matrix.cpp`.
3. Test shape preconditions before extending any algorithm.

4. For numerically sensitive features, prefer tolerance-based comparisons over exact floating-point equality.
5. Keep mathematics and naming aligned: if a helper is library-specific rather than standard, document that fact clearly.

A

Quick Reference

A.1 Public API map

Function or helper	Main mathematical idea	Short usage note
`matrix()`, `matrix(r,c)`	Zero matrix construction	Allocates a matrix of the requested shape.
`get_element`, `ref_element`	Coordinate selection	Direct access to one entry.
`operator+`, `operator-`	Entrywise vector space operations	Requires equal shapes.
`operator*(double)`	Scalar multiplication	Multiplies every entry by the scalar.
`operator*(matrix)`	Composition of linear maps	Requires matching inner dimension.
`row_op`, `row_swap`, `row_multi`	Elementary row operations	Basis of elimination routines.
`col_op`, `col_swap`, `col_multi`	Elementary column operations	Useful for canonical forms.
`echelon`, `gaussian`, `gauss_jordan`	Elimination	Mutates the matrix in place.
`canonical`	Mixed row/column reduction	Produces a more diagonalized pattern.
`rank`	Dimension of row space	Counts nonzero rows after elimination.
`free_variable`	Rank-nullity on square systems	Returns $n - \mathrm{rank}(A)$.
`trace`	Sum of diagonal	Spectral invariant for square matrices.
`diag_product`	Product of diagonal	Equals determinant for triangular matrices.
`det`	Signed volume scaling	Computed through elimination.
`minor`, `cofactor`, `adjoint`	Cofactor expansion machinery	Defined for square matrices.
`transpose`	Reflection across the main diagonal	Swaps rows and columns.
`inverse`	Undoing a linear map	Uses Gauss-Jordan elimination.

Function or helper	Main mathematical idea	Short usage note
`solve`	Solve $Ax = b$	Input must be an augmented matrix.
`norm`, `inner_product`, `angle`	Frobenius geometry	Treats matrices as vectors in entry space.
`matpow`	Fast repeated multiplication	Uses exponentiation by squaring.
`char_poly`	Characteristic polynomial coefficients	Uses Faddeev-LeVerrier recurrence.
`orthogonalize`, `orthonormalize`	Gram-Schmidt process	Returns orthogonal/orthonormal column sets.
`qr_decomp_q`, `qr_decomp_r`	QR decomposition	Returns Q and R factors.
`eigenvalues`	Unshifted QR iteration	Square matrices only; returns diagonal estimates as a column vector.
`check_upper_tri`, `check_lower_tri`	Triangular structure tests	Exact checks on strict triangular entries.
`check_ortho`	Orthogonality test	Checks $A^{-1} = A^{\top}$.
`check_unitary`	Legacy project helper	Not a standard unitary predicate.
`shift`, `scale`, `rotate`, `rot_x`, `rot_y`, `rot_z`	Homogeneous transforms	Angles are in radians.
`vec1` to `vec5`	Small vector constructors	Creates $n \times 1$ column vectors.
`check_lin_comb`	Span-membership check	Tests whether the last vector is a combination of earlier vectors.
`fpg`	Floating-point cleanup	Sets entries with $\lvert x \rvert \leq$ `EPS` to zero.

B
Build This Booklet

B.1 Command line

From the repository root, run:

```
make -C docs pdf
```

The generated PDF is placed in `docs/build/elda_booklet.pdf`.

B.2 Why this format works well

LaTeX is a strong fit for mathematical library documentation because it combines precise equations, technical prose, code listings, page structure, and stable PDF output in one source format. That makes it appropriate for a library like ELDA, where understanding the theory behind each function is part of understanding the software.

C

Complete 80-Function Catalog

C.1 Scope

This chapter gives a dedicated explanation and runnable-style example for every function-level API point. The catalog covers the 79 explicitly declared public functions, plus one compiler-generated constructor (`matrix(const matrix\&)`) that is used in normal C++ value semantics.

C.2 Matrix Construction and Access

C.2.1 `matrix()`

Creates a default 3×3 zero matrix. It is useful as a quick placeholder before assigning explicit entries.

Example

```
matrix a; a.arr = {{1,2,3},{4,5,6},{7,8,9}};
```

C.2.2 `matrix(int r, int c)`

Creates an $r \times c$ zero matrix. Use this constructor when shape is known at runtime or differs from 3×3.

Example

```
matrix b(2, 4); b.arr = {{1,2,3,4},{5,6,7,8}};
```

C.2.3 `matrix(const matrix\&)`

Compiler-generated copy constructor. It copies dimensions and stored entries and is used whenever a matrix is copied by value.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a; // copy construction
```

C.2.4 `get_element(int i, int j)`

Returns the value at row `i`, column `j`. It is a direct accessor and does not perform bounds checks.

Example

```
matrix a(2,2); a.arr = {{10,20},{30,40}}; double x = a.get_element(1, 0);
// 30
```

C.2.5 `ref_element(int i, int j)`

Returns a mutable pointer to one matrix element, enabling in-place updates through dereferencing.

Example

```
matrix a(2,2); *a.ref_element(0, 1) = 7.5;
```

C.2.6 `input()`

Reads all matrix entries from standard input in row-major order. It is useful for interactive console workflows.

Example

```
matrix a(2,2); a.input(); // expects 4 values from stdin
```

C.2.7 `print()`

Prints matrix entries row by row to standard output. It is a diagnostic and display helper.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; a.print();
```

C.3 Arithmetic and Comparison

C.3.1 `operator=(matrix)`

Assigns one matrix to another matrix of the same shape. A dimension mismatch raises an exception in this library.

Example

```
matrix a(2,2), b(2,2); b.arr = {{1,0},{0,1}}; a = b;
```

C.3.2 `operator+(matrix)`

Performs entrywise addition for shape-compatible matrices. It returns a new matrix and leaves operands unchanged.

Example

```
matrix a(2,2), b(2,2); a.arr = {{1,2},{3,4}}; b.arr = {{5,6},{7,8}};
matrix s = a + b;
```

C.3.3 operator-(matrix)

Performs entrywise subtraction for shape-compatible matrices. It returns a new matrix.

Example

```
matrix a(2,2), b(2,2); a.arr = {{5,6},{7,8}}; b.arr = {{1,2},{3,4}};
matrix d = a - b;
```

C.3.4 operator*(matrix)

Computes matrix multiplication when inner dimensions match. This represents composition of linear maps.

Example

```
matrix a(2,3), b(3,1); a.arr = {{1,2,3},{4,5,6}}; b.arr = {{1},{0},{1}};
matrix p = a * b;
```

C.3.5 operator*(double)

Multiplies each entry by a scalar value. It returns a scaled copy.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix k = a * 0.5;
```

C.3.6 operator==(matrix, matrix)

Checks exact entrywise equality via stored values. It is strict and does not use tolerance.

Example

```
matrix a(2,2), b(2,2); a.arr = {{1,2},{3,4}}; b.arr = {{1,2},{3,4}}; bool
eq = (a == b);
```

C.3.7 shape_comp(matrix, matrix)

Checks whether two matrices have identical dimensions, independent of entry values.

Example

```
matrix a(2,3), b(2,3), c(3,2); bool ab = shape_comp(a,b); // true bool ac
= shape_comp(a,c); // false
```

C.3.8 `identity(int n)`

Builds the $n \times n$ identity matrix with ones on the main diagonal and zeros elsewhere.

Example

```
matrix i3 = identity(3);
```

C.4 Elementary Row and Column Operations

C.4.1 `row\_op(int i, double coeff, int j)`

Returns a copy with $R_i \leftarrow R_i + \text{coeff} \cdot R_j$. This is a core elimination primitive.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a.row_op(1, -3.0, 0);
```

C.4.2 `col\_op(int i, double coeff, int j)`

Returns a copy with $C_i \leftarrow C_i + \text{coeff} \cdot C_j$. This supports column-based reductions.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a.col_op(1, -2.0, 0);
```

C.4.3 `row\_swap(int i, int j)`

Returns a copy with two rows exchanged. It is used for pivoting during elimination.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a.row_swap(0, 1);
```

C.4.4 `col\_swap(int i, int j)`

Returns a copy with two columns exchanged. Useful in canonical-form style transformations.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a.col_swap(0, 1);
```

C.4.5 `row\_multi(int i, double factor)`

Returns a copy with row `i` scaled by `factor`. It is used to normalize pivots.

Example

```
matrix a(2,2); a.arr = {{2,4},{3,6}}; matrix b = a.row_multi(0, 0.5);
```

C.4.6 col_multi(int i, double factor)

Returns a copy with column `i` scaled by `factor`.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix b = a.col_multi(1, 3.0);
```

C.4.7 get_row(int r)

Returns a matrix with row `r` preserved and every other row zeroed.

Example

```
matrix a(3,3); a.arr = {{1,2,3},{4,5,6},{7,8,9}}; matrix only_r1 =
a.get_row(1);
```

C.4.8 get_col(int c)

Returns a matrix with column `c` preserved and all other columns zeroed.

Example

```
matrix a(3,3); a.arr = {{1,2,3},{4,5,6},{7,8,9}}; matrix only_c2 =
a.get_col(2);
```

C.4.9 get_row_vec(int r)

Returns row `r` as a $1 \times n$ row vector matrix.

Example

```
matrix a(2,3); a.arr = {{1,2,3},{4,5,6}}; matrix rv = a.get_row_vec(0);
```

C.4.10 get_col_vec(int c)

Returns column `c` as an $m \times 1$ column vector matrix.

Example

```
matrix a(2,3); a.arr = {{1,2,3},{4,5,6}}; matrix cv = a.get_col_vec(1);
```

C.4.11 replace_row(int r, matrix rw)

Replaces row `r` with the first row from `rw`.

Example

```
matrix a(2,3), rw(1,3); a.arr = {{1,2,3},{4,5,6}}; rw.arr = {{9,9,9}};
a.replace_row(1, rw);
```

C.4.12 replace_col(int c, matrix cn)

Replaces column `c` with the first column from `cn`.

Example

```
matrix a(2,2), cn(2,1); a.arr = {{1,2},{3,4}}; cn.arr = {{7},{8}};
a.replace_col(0, cn);
```

C.5 Elimination and System Solving

C.5.1 echelon()

Transforms the matrix in place to row-echelon form and returns row-swap sign information.

Example

```
matrix a(2,2); a.arr = {{2,1},{4,3}}; int s = a.echelon();
```

C.5.2 gaussian()

Performs normalized forward elimination in place and returns row-swap sign information.

Example

```
matrix a(2,2); a.arr = {{3,6},{6,15}}; int s = a.gaussian();
```

C.5.3 gauss_jordan()

Runs Gaussian elimination and clears entries above pivots to reach reduced row-echelon style form.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; int s = a.gauss_jordan();
```

C.5.4 canonical()

Uses row and column elimination toward a canonical/diagonalized structure.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; int s = a.canonical();
```

C.5.5 rank()

Returns matrix rank by counting nonzero rows after elimination.

Example

```
matrix a(2,2); a.arr = {{1,2},{2,4}}; int r = a.rank();
```

C.5.6 free_variable()

For square coefficient matrices, returns `row - rank()`.

Example

```
matrix a(3,3); a.arr = {{1,0,0},{0,1,0},{0,0,0}}; int fv =
a.free_variable();
```

C.5.7 solve()

Solves an augmented $N \times (N+1)$ system by elimination and back substitution.

Example

```
matrix aug(2,3); aug.arr = {{1,1,3},{2,3,8}}; matrix x = aug.solve();
```

C.6 Matrix Invariants and Derived Matrices

C.6.1 trace()

Returns the sum of diagonal entries for square matrices.

Example

```
matrix a(2,2); a.arr = {{2,1},{5,3}}; double t = a.trace();
```

C.6.2 diag_product()

Returns the product of diagonal entries.

Example

```
matrix a(2,2); a.arr = {{2,7},{0,5}}; double p = a.diag_product();
```

C.6.3 det()

Computes determinant via echelon reduction and swap-sign adjustment.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; double d = a.det();
```

C.6.4 minor(int x, int y)

Computes the minor determinant obtained by removing row `x` and column `y`.

Example

```
matrix a(3,3); a.arr = {{1,2,3},{4,5,6},{7,8,9}}; double m =
a.minor(1,1);
```

C.6.5 `cofactor(int x, int y)`

Applies checkerboard sign to the corresponding minor.

Example

```
matrix a(3,3); a.arr = {{1,2,3},{4,5,6},{7,8,9}}; double c =
a.cofactor(0,1);
```

C.6.6 `adjoint()`

Returns the classical adjoint (adjugate), the transpose of the cofactor matrix.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix ad = a.adjoint();
```

C.6.7 `transpose()`

Returns the matrix with rows and columns exchanged.

Example

```
matrix a(2,3); a.arr = {{1,2,3},{4,5,6}}; matrix at = a.transpose();
```

C.6.8 `inverse()`

Computes the inverse of a nonsingular square matrix using Gauss-Jordan elimination.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix inv = a.inverse();
```

C.6.9 `orthogonalize()`

Applies Gram-Schmidt to matrix columns and returns orthogonal output columns.

Example

```
matrix a(3,2); a.arr = {{1,1},{1,0},{0,1}}; matrix q_like =
a.orthogonalize();
```

C.6.10 `orthonormalize()`

Applies Gram-Schmidt and normalizes each resulting column to unit norm.

Example

```
matrix a(3,2); a.arr = {{1,1},{1,0},{0,1}}; matrix q =
a.orthonormalize();
```

C.6.11 qr_decomp_q()

Returns the Q factor from the QR decomposition routine.

Example

```
matrix a(3,2); a.arr = {{1,1},{1,0},{0,1}}; matrix q = a.qr_decomp_q();
```

C.6.12 qr_decomp_r()

Returns the R factor from QR decomposition.

Example

```
matrix a(3,2); a.arr = {{1,1},{1,0},{0,1}}; matrix r = a.qr_decomp_r();
```

C.6.13 check_upper_tri()

Checks whether all strictly lower-triangular entries are exactly zero.

Example

```
matrix u(3,3); u.arr = {{1,2,3},{0,4,5},{0,0,6}}; bool ok =
u.check_upper_tri();
```

C.6.14 check_lower_tri()

Checks whether all strictly upper-triangular entries are exactly zero.

Example

```
matrix l(3,3); l.arr = {{1,0,0},{2,3,0},{4,5,6}}; bool ok =
l.check_lower_tri();
```

C.6.15 char_poly()

Returns characteristic polynomial coefficients using the Faddeev-LeVerrier recurrence.

Example

```
matrix a(2,2); a.arr = {{1,2},{3,4}}; matrix coeff = a.char_poly();
```

C.6.16 eigenvalues()

Estimates eigenvalues of a square matrix using unshifted QR iteration.

Example

```
matrix a(3,3); a.arr = {{3,0,0},{0,2,1},{0,1,2}}; matrix eig =
a.eigenvalues();
```

C.6.17 `norm()`

Returns the Frobenius norm $\|A\|_F$.

Example

```
matrix a(2,2); a.arr = {{1,2},{2,1}}; double n = a.norm();
```

C.7 Global Numerical and Algebra Helpers

C.7.1 `neg\_zero(matrix\& m)`

Replaces signed zero values (-0) with ordinary zero for cleaner output and comparisons.

Example

```
matrix a(1,2); a.arr = {{-0.0, 1.0}}; neg_zero(a);
```

C.7.2 `fpg(matrix\& m)`

Performs floating-point cleanup by setting entries with $|x| \leq$ `EPS` to zero.

Example

```
matrix a(1,2); a.arr = {{1e-8, 2.0}}; fpg(a);
```

C.7.3 `matpow(matrix mat, long long expo)`

Computes A^k with exponentiation by squaring.

Example

```
matrix a(2,2); a.arr = {{1,1},{0,1}}; matrix a5 = matpow(a, 5);
```

C.7.4 `check\_ortho(matrix mat)`

Checks the library criterion $A^\top = A^{-1}$ for orthogonality.

Example

```
matrix r = rotate(PI / 6); bool is_ortho = check_ortho(r);
```

C.7.5 `check\_unitary(matrix mat)`

Legacy helper that compares `transpose()` and `adjoint()` exactly.

Example

```
matrix a = identity(3); bool ok = check_unitary(a);
```

C.7.6 `inner\_product(matrix a, matrix b)`

Returns Frobenius inner product $\langle A, B\rangle = \mathrm{tr}(A^\top B)$.

Example

```
matrix a(2,2), b(2,2); a.arr = {{1,2},{0,1}}; b.arr = {{3,4},{5,6}};
double ip = inner_product(a,b);
```

C.7.7 `angle(matrix a, matrix b)`

Returns angle in radians from the Frobenius inner product formula.

Example

```
matrix a = identity(2); matrix b = identity(2); double th = angle(a,b);
```

C.8 Vector Utility Constructors and Span Checks

C.8.1 `vec1()`

Returns a zero 1×1 column vector.

Example

```
matrix v = vec1();
```

C.8.2 `vec2()`

Returns a zero 2×1 column vector.

Example

```
matrix v = vec2();
```

C.8.3 `vec3()`

Returns a zero 3×1 column vector.

Example

```
matrix v = vec3();
```

C.8.4 `vec4()`

Returns a zero 4×1 column vector.

Example

```
matrix v = vec4();
```

C.8.5 `vec5()`

Returns a zero 5×1 column vector.

Example

```
matrix v = vec5();
```

C.8.6 `vec1(double x)`

Builds a 1×1 column vector with value `x`.

Example

```
matrix v = vec1(3.0);
```

C.8.7 `vec2(double x, double y)`

Builds a 2×1 column vector with entries (x, y).

Example

```
matrix v = vec2(3.0, 4.0);
```

C.8.8 `vec3(double x, double y, double z)`

Builds a 3×1 column vector with entries (x, y, z).

Example

```
matrix v = vec3(1.0, 2.0, 3.0);
```

C.8.9 `vec4(double x, double y, double z, double w)`

Builds a 4×1 column vector, commonly for homogeneous coordinates.

Example

```
matrix p = vec4(1.0, 0.0, 0.0, 1.0);
```

C.8.10 `vec5(double x, double y, double z, double w, double u)`

Builds a 5×1 column vector with five explicit entries.

Example

```
matrix v = vec5(1.0, 2.0, 3.0, 4.0, 5.0);
```

C.8.11 `check\_lin\_comb(matrix m1, matrix m2)`

Tests whether `m2` belongs to span$\{m1\}$ using rank equality.

Example

```
bool in_span = check_lin_comb(vec2(1,0), vec2(2,0));
```

C.8.12 check_lin_comb(matrix m1, matrix m2, matrix m3)

Tests whether `m3` is a linear combination of `m1` and `m2`.

Example

```
bool in_span = check_lin_comb(vec3(1,0,0), vec3(0,1,0), vec3(2,3,0));
```

C.8.13 check_lin_comb(matrix m1, matrix m2, matrix m3, matrix m4)

Tests whether `m4` lies in span$\{m1, m2, m3\}$.

Example

```
bool in_span = check_lin_comb(vec3(1,0,0), vec3(0,1,0), vec3(0,0,1),
vec3(1,2,3));
```

C.8.14 check_lin_comb(matrix m1, matrix m2, matrix m3, matrix m4, matrix m5)

Tests whether `m5` lies in span$\{m1, m2, m3, m4\}$.

Example

```
bool in_span = check_lin_comb(vec4(1,0,0,0), vec4(0,1,0,0), vec4(0,0,1,0),
vec4(0,0,0,1), vec4(2,3,4,5));
```

C.9 2D and 3D Transformation Factories

C.9.1 shift(double dx, double dy)

Builds a 3×3 homogeneous translation matrix for 2D points.

Example

```
matrix t = shift(2.0, 3.0);
```

C.9.2 scale(double kx, double ky)

Builds a 3×3 homogeneous scaling matrix for 2D points.

Example

```
matrix s = scale(2.0, 0.5);
```

C.9.3 `rotate(double angle)`

Builds a 3×3 homogeneous planar rotation matrix in radians.

Example

```
matrix r = rotate(PI / 4);
```

C.9.4 `shift(double dx, double dy, double dz)`

Builds a 4×4 homogeneous translation matrix for 3D points.

Example

```
matrix t = shift(1.0, 2.0, 3.0);
```

C.9.5 `scale(double kx, double ky, double kz)`

Builds a 4×4 homogeneous scaling matrix for 3D points.

Example

```
matrix s = scale(2.0, 2.0, 2.0);
```

C.9.6 `rot\_x(double angle)`

Builds a 4×4 homogeneous rotation matrix around the x-axis.

Example

```
matrix rx = rot_x(PI / 6);
```

C.9.7 `rot\_y(double angle)`

Builds a 4×4 homogeneous rotation matrix around the y-axis.

Example

```
matrix ry = rot_y(PI / 6);
```

C.9.8 `rot\_z(double angle)`

Builds a 4×4 homogeneous rotation matrix around the z-axis.

Example

```
matrix rz = rot_z(PI / 6);
```

Closing Note

The strongest documentation for a numerical library is documentation that respects both mathematics and code. ELDA already has a coherent educational core: matrices, elimination, determinants, inverses, and transforms. This booklet formalizes that core into a reference that can support further work, testing, refactoring, and extension.